FloatMate BDOO

Explicación breve y scripts completos para ejecutar en PostgreSQL

Empieza en Norte > Databases > floatmate_norte > Query Tool. Ejecuta 00, 01, 02 y 03 en orden. Los objetos nuevos se crean dentro de floatmate_objetos.

Cada sección contiene qué hace el script, dónde se ejecuta, cómo debes correrlo y su SQL completo. Copia únicamente el código, sin encabezados, pies de página ni las explicaciones. Si continúa en otra página, incluye todas sus páginas.

Antes de empezar

Abre Query Tool sobre floatmate_norte, activa Auto-commit y confirma que tus tablas anteriores tienen datos. El primer script realiza esta comprobación.

01 se ejecuta una sola vez. 02 puede repetirse. 03 crea una instantánea independiente y repetirlo no duplica la carga ni actualiza los datos posteriores.

05 y 08 se ejecutan completos y terminan con ROLLBACK. 07 se ejecuta por secciones, en la misma pestaña, y también termina con ROLLBACK.

Si una ejecución falló y la pestaña quedó en transacción abortada, ejecuta ROLLBACK; antes de corregir y reintentar. Esto descarta los cambios pendientes de esa transacción.

Orden de ejecución

Script	Uso
00_comprobar_base.sql	Comprobar base y datos
01_modelo_objetos.sql	Crear tipos y clases
02_metodos_y_reglas.sql	Crear métodos y triggers
03_importar_datos_actuales.sql	Cargar datos como objetos
04_consultas_sustentacion.sql	Consultar y sustentar
05_pruebas_automaticas.sql	Probar el módulo con ROLLBACK
06_puente_bdd_opcional.sql	Puente Norte Sur opcional
07_demostracion_guiada.sql	Demostrar el ciclo por secciones
08_crear_objeto_ejemplo.sql	Crear un objeto de ejemplo

El módulo principal también puede instalarse en Sur leyendo sus datos locales. Empieza en Norte. Los scripts 06 y 08 de esta guía están preparados para Norte.

00 Comprobar la base de datos

Archivo: 00_comprobar_base.sql

Qué hace: Comprueba que abriste floatmate_norte o floatmate_sur y que existen las tablas del proyecto anterior. Muestra los registros disponibles; no modifica datos.

Dónde ejecutarlo: Norte > Databases > floatmate_norte > Query Tool.

Cómo ejecutarlo: Ejecuta el script completo. Es el primer paso.

Resultado esperado: Debe indicar que la base es correcta y mostrar registros en vehículos, conductores, rutas, viajes y mantenimientos.

Script completo

```
/* PRIMERO: abrir Query Tool en Norte > floatmate_norte.
   Tambien sirve en Sur > floatmate_sur si desea instalar el modulo alli.
   Solo consulta. No crea bases, usuarios ni roles. */
SELECT current_database ( ) AS base_actual , current_user AS usuario ,
       current_setting ( 'port' ) AS puerto , version ( ) AS version;
DO $$
DECLARE t text;
BEGIN
  IF current_database ( ) NOT IN ( 'floatmate_norte' , 'floatmate_sur' ) THEN
    RAISE EXCEPTION 'Abra Query Tool sobre floatmate_norte o floatmate_sur; base actual: %'
      , current_database ( ) ;
  END IF;
  FOREACH t IN ARRAY ARRAY[ 'vehiculos' , 'conductores' , 'rutas' , 'viajes' ,
    'mantenimientos' , 'controles_salida' ] LOOP
    IF to_regclass ( 'floatmate.' || t ) IS NULL THEN
      RAISE EXCEPTION
        'Falta floatmate.%. Termine primero el modelo y los datos de la actividad BDD.' , t;
    END IF;
  END LOOP;
  RAISE NOTICE 'Base correcta. Siga con 01_modelo_objetos.sql.' ;
END $$;
SELECT 'vehiculos' AS tabla , count ( * ) AS registros FROM floatmate.vehiculos
UNION ALL SELECT 'conductores' , count ( * ) FROM floatmate.conductores
UNION ALL SELECT 'rutas' , count ( * ) FROM floatmate.rutas
UNION ALL SELECT 'viajes' , count ( * ) FROM floatmate.viajes
UNION ALL SELECT 'mantenimientos' , count ( * ) FROM floatmate.mantenimientos;
```

Fin del script 00_comprobar_base.sql

01 Crear el modelo de objetos

Archivo: 01_modelo_objetos.sql

Qué hace: Crea floatmate_objetos, tipos compuestos, dominios, enumeraciones y tablas. Define Vehículo y sus tres subclases, la identidad UUID y las relaciones de viajes y mantenimientos.

Dónde ejecutarlo: Norte > floatmate_norte > Query Tool, después del script 00.

Cómo ejecutarlo: Ejecuta completo una sola vez. Si ya se instaló correctamente, continúa con 02.

Resultado esperado: Después de actualizar Schemas, aparece floatmate_objetos. Todavía no se han cargado los datos.

Script completo

```
/* FLOATMATE - BDOO / modelo objeto-relacional.
   Ejecutar completo en floatmate_norte, despues de 00.
   Instalacion inicial: este archivo se ejecuta UNA VEZ.
   No modifica floatmate, floatmate_remoto, floatmate_global ni floatmate_programacion. */
BEGIN;
DO $$
BEGIN
  IF to_regclass ( 'floatmate.vehiculos' ) IS NULL THEN
    RAISE EXCEPTION 'Esta base no tiene floatmate.vehiculos. Abra la base correcta.' ;
  END IF;
  IF to_regnamespace ( 'floatmate_objetos' ) IS NOT NULL THEN
    RAISE EXCEPTION
      'floatmate_objetos ya existe. No se borra nada. Si ya instalo 01, siga con 02.' ;
  END IF;
END $$;
CREATE SCHEMA floatmate_objetos;
COMMENT ON SCHEMA floatmate_objetos IS
  'Actividad BDOO: clases, objetos persistentes, UUID, composicion, herencia y metodos.' ;

CREATE TYPE floatmate_objetos.clase_vehiculo AS ENUM ( 'BUS_URBANO' , 'BUS_INTERMUNICIPAL' ,
  'MICROBUS' ) ;
CREATE TYPE floatmate_objetos.estado_vehiculo AS ENUM ( 'DISPONIBLE' , 'EN_RUTA' ,
  'MANTENIMIENTO' , 'FUERA_SERVICIO' ) ;
CREATE TYPE floatmate_objetos.estado_viaje AS ENUM ( 'PROGRAMADO' , 'EN_CURSO' ,
  'FINALIZADO' , 'CANCELADO' ) ;
CREATE TYPE floatmate_objetos.t_contacto AS ( telefono text , correo text ) ;
CREATE TYPE floatmate_objetos.t_licencia AS ( numero text , categoria text , vence date ) ;
CREATE TYPE floatmate_objetos.t_ficha AS (
  marca text , modelo smallint , capacidad smallint ,
  kilometraje numeric ( 12 , 2 ) , proximo_mantenimiento_km numeric ( 12 , 2 )
) ;
/* Los CREATE TYPE compuestos no aceptan CHECK internos.
   El dominio valida incluso los valores compuestos usados fuera de una tabla. */
CREATE DOMAIN floatmate_objetos.d_contacto AS floatmate_objetos.t_contacto
  CHECK ( ( VALUE ) .telefono IS NOT NULL AND length ( trim ( ( VALUE ) .telefono ) ) >= 7
    AND ( ( VALUE ) .correo IS NULL OR position ( '@' IN ( VALUE ) .correo ) > 1 ) ) ;
CREATE DOMAIN floatmate_objetos.d_licencia AS floatmate_objetos.t_licencia
  CHECK ( ( VALUE ) .numero IS NOT NULL AND length ( trim ( ( VALUE ) .numero ) ) > 0
    AND ( VALUE ) .categoria IS NOT NULL AND ( VALUE ) .categoria IN ( 'C1' , 'C2' , 'C3' )
    AND ( VALUE ) .vence IS NOT NULL ) ;
CREATE DOMAIN floatmate_objetos.d_ficha AS floatmate_objetos.t_ficha
  CHECK ( ( VALUE ) .marca IS NOT NULL AND length ( trim ( ( VALUE ) .marca ) ) > 0
    AND ( VALUE ) .modelo IS NOT NULL AND ( VALUE ) .modelo BETWEEN 2000 AND 2035
    AND ( VALUE ) .capacidad IS NOT NULL AND ( VALUE ) .capacidad BETWEEN 10 AND 80
    AND ( VALUE ) .kilometraje IS NOT NULL AND ( VALUE ) .kilometraje >= 0
```

```
    AND ( VALUE ) .proximo_mantenimiento_km IS NOT NULL AND ( VALUE )
      .proximo_mantenimiento_km >= 0 ) ;

/* Registro central: la unicidad del padre INHERITS no cubre sus hijos.
   Cada objeto vehiculo se identifica aqui; los hijos hacen referencia a este registro. */
CREATE TABLE floatmate_objetos.identidad_vehiculo (
  oid uuid PRIMARY KEY DEFAULT gen_random_uuid ( ) ,
  terminal_codigo varchar ( 3 ) NOT NULL CHECK ( terminal_codigo IN ( 'NOR' , 'SUR' ) ) ,
  id_relacional bigint ,
  clase floatmate_objetos.clase_vehiculo NOT NULL ,
  placa varchar ( 6 ) NOT NULL UNIQUE CHECK ( placa ~ '^[A-Z]{3}[0-9]{3}$' ) ,
  creado_en timestamptz NOT NULL DEFAULT CURRENT_TIMESTAMP ,
  UNIQUE ( terminal_codigo , id_relacional ) ,
  UNIQUE ( oid , terminal_codigo ) ,
  UNIQUE ( oid , clase , terminal_codigo , placa )
) ;
COMMENT ON COLUMN floatmate_objetos.identidad_vehiculo.oid IS
    'OID del modelo de objetos: UUID estable e inmutable. No es el tipo oid del catalogo '
    'PostgreSQL.' ;

CREATE TABLE floatmate_objetos.vehiculo (
  oid uuid NOT NULL ,
  terminal_codigo varchar ( 3 ) NOT NULL CHECK ( terminal_codigo IN ( 'NOR' , 'SUR' ) ) ,
  clase floatmate_objetos.clase_vehiculo NOT NULL ,
  placa varchar ( 6 ) NOT NULL CHECK ( placa ~ '^[A-Z]{3}[0-9]{3}$' ) ,
  ficha floatmate_objetos.d_ficha NOT NULL ,
  estado floatmate_objetos.estado_vehiculo NOT NULL DEFAULT 'DISPONIBLE' ,
  equipamiento text[] NOT NULL DEFAULT ARRAY[ 'BOTIQUIN' , 'EXTINTOR' ]::text[] ,
  CHECK ( cardinality ( equipamiento ) > 0 AND array_position ( equipamiento , NULL ) IS
    NULL )
) ;
COMMENT ON TABLE floatmate_objetos.vehiculo IS
    'Superclase conceptual abstracta. SELECT incluye objetos de los tres hijos; ONLY '
    'consulta solo el padre.' ;

CREATE TABLE floatmate_objetos.bus_urbano (
  clase floatmate_objetos.clase_vehiculo DEFAULT 'BUS_URBANO' ,
  puertas smallint NOT NULL DEFAULT 2 CHECK ( puertas BETWEEN 1 AND 4 ) ,
  accesible boolean NOT NULL DEFAULT true ,
  PRIMARY KEY ( oid ) , UNIQUE ( placa ) ,
  CHECK ( clase = 'BUS_URBANO' ) ,
  FOREIGN KEY ( oid , clase , terminal_codigo , placa ) REFERENCES
    floatmate_objetos.identidad_vehiculo ( oid , clase , terminal_codigo , placa )
    DEFERRABLE INITIALLY DEFERRED
) INHERITS ( floatmate_objetos.vehiculo ) ;
CREATE TABLE floatmate_objetos.bus_intermunicipal (
  clase floatmate_objetos.clase_vehiculo DEFAULT 'BUS_INTERMUNICIPAL' ,
  aire_acondicionado boolean NOT NULL DEFAULT true ,
  bodega_m3 numeric ( 5 , 2 ) NOT NULL DEFAULT 8 CHECK ( bodega_m3 >= 0 ) ,
  PRIMARY KEY ( oid ) , UNIQUE ( placa ) ,
  CHECK ( clase = 'BUS_INTERMUNICIPAL' ) ,
  FOREIGN KEY ( oid , clase , terminal_codigo , placa ) REFERENCES
    floatmate_objetos.identidad_vehiculo ( oid , clase , terminal_codigo , placa )
    DEFERRABLE INITIALLY DEFERRED
) INHERITS ( floatmate_objetos.vehiculo ) ;
CREATE TABLE floatmate_objetos.microbus (
  clase floatmate_objetos.clase_vehiculo DEFAULT 'MICROBUS' ,
  servicio_especial boolean NOT NULL DEFAULT false ,
  PRIMARY KEY ( oid ) , UNIQUE ( placa ) ,
  CHECK ( clase = 'MICROBUS' ) ,
  FOREIGN KEY ( oid , clase , terminal_codigo , placa ) REFERENCES
    floatmate_objetos.identidad_vehiculo ( oid , clase , terminal_codigo , placa )
```

```
    DEFERRABLE INITIALLY DEFERRED
) INHERITS ( floatmate_objetos.vehiculo ) ;

CREATE TABLE floatmate_objetos.conductor (
  oid uuid PRIMARY KEY DEFAULT gen_random_uuid ( ) ,
  terminal_codigo varchar ( 3 ) NOT NULL CHECK ( terminal_codigo IN ( 'NOR' , 'SUR' ) ) ,
  id_relacional bigint ,
  documento text NOT NULL UNIQUE ,
  nombre text NOT NULL CHECK ( length ( trim ( nombre ) ) > 0 ) ,
  contacto floatmate_objetos.d_contacto NOT NULL ,
  licencia floatmate_objetos.d_licencia NOT NULL ,
  estado text NOT NULL CHECK ( estado IN ( 'DISPONIBLE' , 'EN_VIAJE' , 'INACTIVO' ) ) ,
  UNIQUE ( terminal_codigo , id_relacional ) ,
  UNIQUE ( oid , terminal_codigo )
) ;
CREATE TABLE floatmate_objetos.ruta (
  oid uuid PRIMARY KEY DEFAULT gen_random_uuid ( ) ,
  terminal_codigo varchar ( 3 ) NOT NULL CHECK ( terminal_codigo IN ( 'NOR' , 'SUR' ) ) ,
  id_relacional bigint ,
  nombre text NOT NULL ,
  origen text NOT NULL , destino text NOT NULL CHECK ( destino <> origen ) ,
  distancia_km numeric ( 8 , 2 ) NOT NULL CHECK ( distancia_km > 0 ) ,
  paradas text[] NOT NULL CHECK ( cardinality ( paradas ) >= 2 AND array_position ( paradas
    , NULL ) IS NULL ) ,
  activa boolean NOT NULL DEFAULT true ,
  UNIQUE ( terminal_codigo , id_relacional ) ,
  UNIQUE ( oid , terminal_codigo )
) ;
CREATE TABLE floatmate_objetos.viaje (
  oid uuid PRIMARY KEY DEFAULT gen_random_uuid ( ) ,
  terminal_codigo varchar ( 3 ) NOT NULL CHECK ( terminal_codigo IN ( 'NOR' , 'SUR' ) ) ,
  id_relacional bigint ,
  vehiculo_oid uuid NOT NULL ,
  conductor_oid uuid NOT NULL ,
  ruta_oid uuid NOT NULL ,
  fecha_programada timestamptz NOT NULL ,
  fecha_salida timestamptz , fecha_llegada timestamptz ,
  pasajeros smallint NOT NULL CHECK ( pasajeros >= 0 ) ,
  estado floatmate_objetos.estado_viaje NOT NULL DEFAULT 'PROGRAMADO' ,
  expediente jsonb NOT NULL DEFAULT '{}' ::jsonb CHECK ( jsonb_typeof ( expediente ) =
    'object' ) ,
  UNIQUE ( terminal_codigo , id_relacional ) ,
  FOREIGN KEY ( vehiculo_oid , terminal_codigo ) REFERENCES
    floatmate_objetos.identidad_vehiculo ( oid , terminal_codigo ) ,
  FOREIGN KEY ( conductor_oid , terminal_codigo ) REFERENCES floatmate_objetos.conductor (
    oid , terminal_codigo ) ,
  FOREIGN KEY ( ruta_oid , terminal_codigo ) REFERENCES floatmate_objetos.ruta ( oid ,
    terminal_codigo ) ,
  CHECK ( fecha_llegada IS NULL OR ( fecha_salida IS NOT NULL AND fecha_llegada >=
    fecha_salida ) )
) ;
CREATE UNIQUE INDEX ux_obj_viaje_vehiculo_hora ON floatmate_objetos.viaje ( vehiculo_oid ,
  fecha_programada )
  WHERE estado <> 'CANCELADO' ;
CREATE UNIQUE INDEX ux_obj_viaje_conductor_hora ON floatmate_objetos.viaje ( conductor_oid ,
  fecha_programada )
  WHERE estado <> 'CANCELADO' ;
CREATE TABLE floatmate_objetos.mantenimiento (
  oid uuid PRIMARY KEY DEFAULT gen_random_uuid ( ) ,
  terminal_codigo varchar ( 3 ) NOT NULL CHECK ( terminal_codigo IN ( 'NOR' , 'SUR' ) ) ,
  id_relacional bigint ,
  vehiculo_oid uuid NOT NULL ,
```

```
  tipo text NOT NULL CHECK ( tipo IN ( 'PREVENTIVO' , 'CORRECTIVO' ) ) ,
  fecha_ingreso date NOT NULL ,
  costo numeric ( 12 , 2 ) NOT NULL CHECK ( costo >= 0 ) ,
  detalle jsonb NOT NULL CHECK ( jsonb_typeof ( detalle ) = 'object' ) ,
  UNIQUE ( terminal_codigo , id_relacional ) ,
  FOREIGN KEY ( vehiculo_oid , terminal_codigo ) REFERENCES
    floatmate_objetos.identidad_vehiculo ( oid , terminal_codigo )
) ;
CREATE TABLE floatmate_objetos.auditoria (
  evento_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY ,
  fecha timestamptz NOT NULL DEFAULT CURRENT_TIMESTAMP ,
  usuario text NOT NULL DEFAULT CURRENT_USER ,
  tabla text NOT NULL ,
  operacion text NOT NULL CHECK ( operacion IN ( 'INSERT' , 'UPDATE' , 'DELETE' ) ) ,
  objeto_oid uuid NOT NULL ,
  anterior jsonb , nuevo jsonb
) ;
CREATE TABLE floatmate_objetos.instalacion (
  modulo text PRIMARY KEY ,
  version text NOT NULL ,
  datos_importados_en timestamptz
) ;
INSERT INTO floatmate_objetos.instalacion VALUES ( 'BDOO' , '1.0' , NULL ) ;
COMMIT;
SELECT 'Modelo creado. Siga con 02_metodos_y_reglas.sql.' AS siguiente_paso;
```

Fin del script 01_modelo_objetos.sql

02 Crear métodos y reglas

Archivo: 02_metodos_y_reglas.sql

Qué hace: Crea funciones para describir objetos, calcular costos y recuperar JSON; procedimientos para programar, aprobar, iniciar y cerrar viajes; vistas y triggers de identidad, validación y auditoría.

Dónde ejecutarlo: Norte > floatmate_norte > Query Tool, después del script 01.

Cómo ejecutarlo: Ejecuta completo. Puede repetirse para reemplazar los métodos y triggers del módulo.

Resultado esperado: Aparecen funciones, procedimientos y vistas en floatmate_objetos. Los avisos de triggers inexistentes en la primera instalación son normales.

Script completo

```
/* Ejecutar completo en la MISMA base donde ejecuto 01.
   Se puede volver a ejecutar: reemplaza funciones y triggers del modulo. */
BEGIN;
CREATE OR REPLACE FUNCTION floatmate_objetos.fn_contacto ( p_telefono text , p_correo text
  DEFAULT NULL )
RETURNS floatmate_objetos.d_contacto LANGUAGE sql IMMUTABLE
SET search_path TO pg_catalog AS $$
 SELECT ROW ( p_telefono , p_correo )
   ::floatmate_objetos.t_contacto::floatmate_objetos.d_contacto;
$$;

CREATE OR REPLACE FUNCTION floatmate_objetos.fn_km_restantes ( p_ficha
  floatmate_objetos.t_ficha )
RETURNS numeric LANGUAGE sql IMMUTABLE STRICT SET search_path TO pg_catalog AS $$
 SELECT greatest ( ( p_ficha ) .proximo_mantenimiento_km - ( p_ficha ) .kilometraje , 0 ) ;
$$;

/* Sobrecarga: mismo nombre, diferentes tipos de objeto de entrada.
   Son funciones externas sobre tipos de fila, no metodos miembros de una clase. */
CREATE OR REPLACE FUNCTION floatmate_objetos.fn_describir ( p_obj
  floatmate_objetos.bus_urbano )
RETURNS text LANGUAGE sql IMMUTABLE STRICT SET search_path TO pg_catalog AS $$
 SELECT format ( 'Bus urbano %s: %s puertas; accesible: %s' , ( p_obj ) .placa , ( p_obj )
   .puertas , ( p_obj ) .accesible ) ;
$$;
CREATE OR REPLACE FUNCTION floatmate_objetos.fn_describir ( p_obj
  floatmate_objetos.bus_intermunicipal )
RETURNS text LANGUAGE sql IMMUTABLE STRICT SET search_path TO pg_catalog AS $$
 SELECT format ( 'Bus intermunicipal %s: bodega %s m3; aire: %s' , ( p_obj ) .placa , (
   p_obj ) .bodega_m3 , ( p_obj ) .aire_acondicionado ) ;
$$;
CREATE OR REPLACE FUNCTION floatmate_objetos.fn_describir ( p_obj floatmate_objetos.microbus
  )
RETURNS text LANGUAGE sql IMMUTABLE STRICT SET search_path TO pg_catalog AS $$
 SELECT format ( 'Microbus %s: servicio especial: %s' , ( p_obj ) .placa , ( p_obj )
   .servicio_especial ) ;
$$;
/* Tarifas ficticias COP/km para demostrar comportamiento por clase; no tarifas comerciales.
   */
CREATE OR REPLACE FUNCTION floatmate_objetos.fn_costo ( p_obj floatmate_objetos.bus_urbano ,
  p_km numeric )
RETURNS numeric LANGUAGE plpgsql IMMUTABLE STRICT SET search_path TO pg_catalog AS $$
BEGIN
 IF p_km <= 0 THEN RAISE EXCEPTION 'La distancia debe ser positiva' ; END IF;
 RETURN round ( p_km * 1400 , 2 ) ;
```

```
END $$;
CREATE OR REPLACE FUNCTION floatmate_objetos.fn_costo ( p_obj
  floatmate_objetos.bus_intermunicipal , p_km numeric )
RETURNS numeric LANGUAGE plpgsql IMMUTABLE STRICT SET search_path TO pg_catalog AS $$
BEGIN
 IF p_km <= 0 THEN RAISE EXCEPTION 'La distancia debe ser positiva' ; END IF;
 RETURN round ( p_km * ( 2200 + CASE WHEN ( p_obj ) .aire_acondicionado THEN 200 ELSE 0 END
   ) , 2 ) ;
END $$;
CREATE OR REPLACE FUNCTION floatmate_objetos.fn_costo ( p_obj floatmate_objetos.microbus ,
  p_km numeric )
RETURNS numeric LANGUAGE plpgsql IMMUTABLE STRICT SET search_path TO pg_catalog AS $$
BEGIN
 IF p_km <= 0 THEN RAISE EXCEPTION 'La distancia debe ser positiva' ; END IF;
 RETURN round ( p_km * 1000 , 2 ) ;
END $$;

CREATE OR REPLACE FUNCTION floatmate_objetos.fn_vehiculo_json ( p_oid uuid )
RETURNS jsonb LANGUAGE sql STABLE STRICT SET search_path TO pg_catalog AS $$
 SELECT to_jsonb ( b ) || jsonb_build_object ( 'clase_fisica' , 'bus_urbano' , 'descripcion'
   , floatmate_objetos.fn_describir ( b ) )
 FROM floatmate_objetos.bus_urbano b WHERE b.oid=p_oid
 UNION ALL
 SELECT to_jsonb ( b ) || jsonb_build_object ( 'clase_fisica' , 'bus_intermunicipal' ,
   'descripcion' , floatmate_objetos.fn_describir ( b ) )
 FROM floatmate_objetos.bus_intermunicipal b WHERE b.oid=p_oid
 UNION ALL
 SELECT to_jsonb ( b ) || jsonb_build_object ( 'clase_fisica' , 'microbus' , 'descripcion' ,
   floatmate_objetos.fn_describir ( b ) )
 FROM floatmate_objetos.microbus b WHERE b.oid=p_oid;
$$;
CREATE OR REPLACE FUNCTION floatmate_objetos.fn_viaje_json ( p_oid uuid )
RETURNS jsonb LANGUAGE sql STABLE STRICT SET search_path TO pg_catalog AS $$
 SELECT to_jsonb ( v ) || jsonb_build_object (
   'vehiculo' , floatmate_objetos.fn_vehiculo_json ( v.vehiculo_oid ) ,
   'conductor' , to_jsonb ( c ) , 'ruta' , to_jsonb ( r ) )
 FROM floatmate_objetos.viaje v
 JOIN floatmate_objetos.conductor c ON c.oid=v.conductor_oid
 JOIN floatmate_objetos.ruta r ON r.oid=v.ruta_oid WHERE v.oid=p_oid;
$$;
CREATE OR REPLACE VIEW floatmate_objetos.v_flota AS
 SELECT v.oid , v.terminal_codigo , v.clase , v.placa , v.estado ,
   ( v.ficha ) .marca AS marca , ( v.ficha ) .capacidad AS capacidad ,
   ( v.ficha ) .kilometraje AS kilometraje ,
   floatmate_objetos.fn_km_restantes ( v.ficha::floatmate_objetos.t_ficha ) AS
     km_para_mantenimiento ,
   v.tableoid::regclass::text AS clase_fisica
 FROM floatmate_objetos.vehiculo v;
CREATE OR REPLACE VIEW floatmate_objetos.v_comportamiento AS
 SELECT b.oid , b.placa , b.clase , floatmate_objetos.fn_describir ( b ) AS descripcion ,
        floatmate_objetos.fn_costo ( b , 10::numeric ) AS costo_demo_10_km FROM
          floatmate_objetos.bus_urbano b
 UNION ALL
 SELECT b.oid , b.placa , b.clase , floatmate_objetos.fn_describir ( b ) ,
   floatmate_objetos.fn_costo ( b , 10::numeric )
 FROM floatmate_objetos.bus_intermunicipal b
 UNION ALL
 SELECT b.oid , b.placa , b.clase , floatmate_objetos.fn_describir ( b ) ,
   floatmate_objetos.fn_costo ( b , 10::numeric )
 FROM floatmate_objetos.microbus b;

/* OID y correspondencia con la fuente son inmutables. */
```

```
CREATE OR REPLACE FUNCTION floatmate_objetos.fn_trg_identidad_inmutable ( )
RETURNS trigger LANGUAGE plpgsql SET search_path TO pg_catalog AS $$
BEGIN
 IF NEW.oid IS DISTINCT FROM OLD.oid THEN
   RAISE EXCEPTION 'El OID es inmutable: no puede cambiar la identidad del objeto' ;
 END IF;
 IF to_jsonb ( NEW ) -> 'id_relacional' IS DISTINCT FROM to_jsonb ( OLD ) -> 'id_relacional'
    OR NEW.terminal_codigo IS DISTINCT FROM OLD.terminal_codigo
    OR to_jsonb ( NEW ) -> 'clase' IS DISTINCT FROM to_jsonb ( OLD ) -> 'clase' THEN
   RAISE EXCEPTION 'No puede cambiar la terminal, clase o referencia de origen del objeto' ;
 END IF;
 RETURN NEW;
END $$;
CREATE OR REPLACE FUNCTION floatmate_objetos.fn_trg_padre_abstracto ( )
RETURNS trigger LANGUAGE plpgsql SET search_path TO pg_catalog AS $$
BEGIN
 RAISE EXCEPTION
   'Vehiculo es la superclase abstracta: inserte en bus_urbano, bus_intermunicipal o '
   'microbus' ;
END $$;
DROP TRIGGER IF EXISTS tr_padre_abstracto ON floatmate_objetos.vehiculo;
CREATE TRIGGER tr_padre_abstracto BEFORE INSERT ON floatmate_objetos.vehiculo
 FOR EACH ROW EXECUTE FUNCTION floatmate_objetos.fn_trg_padre_abstracto ( ) ;

/* Al COMMIT cada identidad debe tener exactamente una instancia de una subclase.
   Esta regla resuelve el problema de referenciar hijos de INHERITS. */
CREATE OR REPLACE FUNCTION floatmate_objetos.fn_trg_instancia_completa ( )
RETURNS trigger LANGUAGE plpgsql SET search_path TO pg_catalog AS $$
DECLARE id uuid; n bigint;
BEGIN
 id := CASE WHEN TG_OP= 'DELETE' THEN OLD.oid ELSE NEW.oid END;
 IF EXISTS ( SELECT 1 FROM floatmate_objetos.identidad_vehiculo i WHERE i.oid=id ) THEN
   SELECT count ( * ) INTO n FROM floatmate_objetos.vehiculo v WHERE v.oid=id;
   IF n <> 1 THEN RAISE EXCEPTION
     'La identidad % debe tener exactamente una instancia; tiene %' , id , n; END IF;
 END IF;
 RETURN NULL;
END $$;
CREATE OR REPLACE FUNCTION floatmate_objetos.fn_trg_auditar ( )
RETURNS trigger LANGUAGE plpgsql SET search_path TO pg_catalog AS $$
BEGIN
 INSERT INTO floatmate_objetos.auditoria ( tabla , operacion , objeto_oid , anterior , nuevo
   )
 VALUES ( TG_TABLE_NAME , TG_OP , CASE WHEN TG_OP= 'DELETE' THEN OLD.oid ELSE NEW.oid END ,
        CASE WHEN TG_OP IN ( 'UPDATE' , 'DELETE' ) THEN to_jsonb ( OLD ) END ,
        CASE WHEN TG_OP IN ( 'INSERT' , 'UPDATE' ) THEN to_jsonb ( NEW ) END ) ;
 RETURN NULL;
END $$;
DO $$
DECLARE t text;
BEGIN
 FOREACH t IN ARRAY ARRAY[ 'identidad_vehiculo' , 'bus_urbano' , 'bus_intermunicipal' ,
   'microbus' , 'conductor' , 'ruta' , 'viaje' , 'mantenimiento' ] LOOP
   EXECUTE format ( 'DROP TRIGGER IF EXISTS tr_oid_inmutable ON floatmate_objetos.%I' , t )
     ;
   EXECUTE format (
     'CREATE TRIGGER tr_oid_inmutable BEFORE UPDATE ON floatmate_objetos.%I FOR EACH ROW '
     'EXECUTE FUNCTION floatmate_objetos.fn_trg_identidad_inmutable()' , t ) ;
   EXECUTE format ( 'DROP TRIGGER IF EXISTS tr_auditoria ON floatmate_objetos.%I' , t ) ;
   EXECUTE format (
     'CREATE TRIGGER tr_auditoria AFTER INSERT OR UPDATE OR DELETE ON floatmate_objetos.%I '
     'FOR EACH ROW EXECUTE FUNCTION floatmate_objetos.fn_trg_auditar()' , t ) ;
```

```
 END LOOP;
 FOREACH t IN ARRAY ARRAY[ 'identidad_vehiculo' , 'bus_urbano' , 'bus_intermunicipal' ,
   'microbus' ] LOOP
   EXECUTE format ( 'DROP TRIGGER IF EXISTS tr_instancia_completa ON floatmate_objetos.%I' ,
     t ) ;
   EXECUTE format (
     'CREATE CONSTRAINT TRIGGER tr_instancia_completa AFTER INSERT OR UPDATE OR DELETE ON '
     'floatmate_objetos.%I DEFERRABLE INITIALLY DEFERRED FOR EACH ROW EXECUTE FUNCTION '
     'floatmate_objetos.fn_trg_instancia_completa()' , t ) ;
 END LOOP;
END $$;

CREATE OR REPLACE FUNCTION floatmate_objetos.fn_trg_validar_viaje ( )
RETURNS trigger LANGUAGE plpgsql SET search_path TO pg_catalog AS $$
DECLARE v floatmate_objetos.vehiculo; c floatmate_objetos.conductor; r
  floatmate_objetos.ruta;
BEGIN
 SELECT b.* INTO v FROM floatmate_objetos.vehiculo b WHERE b.oid=NEW.vehiculo_oid FOR
   UPDATE;
 IF NOT FOUND THEN RAISE EXCEPTION 'No existe el objeto vehiculo' ; END IF;
 SELECT b.* INTO c FROM floatmate_objetos.conductor b WHERE b.oid=NEW.conductor_oid FOR
   UPDATE;
 IF NOT FOUND THEN RAISE EXCEPTION 'No existe el objeto conductor' ; END IF;
 SELECT b.* INTO r FROM floatmate_objetos.ruta b WHERE b.oid=NEW.ruta_oid;
 IF NOT FOUND THEN RAISE EXCEPTION 'No existe el objeto ruta' ; END IF;
 IF NEW.terminal_codigo <> v.terminal_codigo OR NEW.terminal_codigo <> c.terminal_codigo OR
   NEW.terminal_codigo <> r.terminal_codigo THEN
   RAISE EXCEPTION 'Todos los objetos del viaje deben pertenecer a la misma terminal' ;
 END IF;
 IF NEW.pasajeros > ( v.ficha ) .capacidad THEN RAISE EXCEPTION
   'Pasajeros superan la capacidad del objeto vehiculo' ; END IF;
 IF TG_OP= 'UPDATE' THEN
   IF OLD.estado IN ( 'EN_CURSO' , 'FINALIZADO' , 'CANCELADO' ) AND
     ( NEW.vehiculo_oid , NEW.conductor_oid , NEW.ruta_oid , NEW.pasajeros ,
       NEW.fecha_programada )
     IS DISTINCT FROM ( OLD.vehiculo_oid , OLD.conductor_oid , OLD.ruta_oid , OLD.pasajeros
       , OLD.fecha_programada ) THEN
     RAISE EXCEPTION 'No puede reasignar un viaje iniciado, finalizado o cancelado' ;
   END IF;
   IF NEW.estado IS DISTINCT FROM OLD.estado THEN
     IF NOT ( ( OLD.estado= 'PROGRAMADO' AND NEW.estado IN ( 'EN_CURSO' , 'CANCELADO' ) ) OR
              ( OLD.estado= 'EN_CURSO' AND NEW.estado= 'FINALIZADO' ) ) THEN
       RAISE EXCEPTION 'Transicion de estado no permitida: % -> %' , OLD.estado ,
         NEW.estado;
     END IF;
   END IF;
 END IF;
 IF NEW.estado= 'EN_CURSO' AND ( TG_OP= 'INSERT' OR OLD.estado IS DISTINCT FROM NEW.estado )
   THEN
   IF v.estado <> 'DISPONIBLE' OR c.estado <> 'DISPONIBLE' OR NOT r.activa THEN
     RAISE EXCEPTION 'Vehiculo/conductor no disponible o ruta inactiva' ;
   END IF;
   IF ( c.licencia ) .vence < NEW.fecha_salida::date THEN RAISE EXCEPTION
     'Licencia vencida para la fecha de salida' ; END IF;
   IF NEW.fecha_salida IS NULL OR NEW.expediente->> 'control_salida' IS DISTINCT FROM
     'APROBADO'
      OR coalesce ( length ( trim ( NEW.expediente->> 'ticket' ) ) , 0 ) =0 THEN
     RAISE EXCEPTION 'Para iniciar necesita fecha de salida, control APROBADO y ticket' ;
   END IF;
 END IF;
 RETURN NEW;
END $$;
```

```
DROP TRIGGER IF EXISTS tr_validar_viaje ON floatmate_objetos.viaje;
CREATE TRIGGER tr_validar_viaje BEFORE INSERT OR UPDATE ON floatmate_objetos.viaje
 FOR EACH ROW EXECUTE FUNCTION floatmate_objetos.fn_trg_validar_viaje ( ) ;

CREATE OR REPLACE PROCEDURE floatmate_objetos.sp_programar_viaje (
 p_vehiculo uuid , p_conductor uuid , p_ruta uuid , p_fecha timestamptz , p_pasajeros
   smallint )
LANGUAGE plpgsql SET search_path TO pg_catalog AS $$
DECLARE v floatmate_objetos.vehiculo; c floatmate_objetos.conductor; r
  floatmate_objetos.ruta; id uuid;
BEGIN
 SELECT b.* INTO STRICT v FROM floatmate_objetos.vehiculo b WHERE b.oid=p_vehiculo FOR
   UPDATE;
 SELECT b.* INTO STRICT c FROM floatmate_objetos.conductor b WHERE b.oid=p_conductor FOR
   UPDATE;
 SELECT b.* INTO STRICT r FROM floatmate_objetos.ruta b WHERE b.oid=p_ruta;
 IF v.estado <> 'DISPONIBLE' OR c.estado <> 'DISPONIBLE' OR NOT r.activa THEN
   RAISE EXCEPTION 'Vehiculo/conductor no disponible o ruta inactiva' ;
 END IF;
 INSERT INTO floatmate_objetos.viaje ( terminal_codigo , vehiculo_oid , conductor_oid ,
   ruta_oid , fecha_programada , pasajeros )
 VALUES ( v.terminal_codigo , p_vehiculo , p_conductor , p_ruta , p_fecha , p_pasajeros )
   RETURNING oid INTO id;
 RAISE NOTICE 'Objeto viaje creado: %' , id;
EXCEPTION WHEN no_data_found THEN RAISE EXCEPTION
  'No existe alguno de los objetos requeridos' ;
END $$;
CREATE OR REPLACE PROCEDURE floatmate_objetos.sp_aprobar_control ( p_viaje uuid , p_ticket
  text )
LANGUAGE plpgsql SET search_path TO pg_catalog AS $$
BEGIN
 IF p_ticket IS NULL OR length ( trim ( p_ticket ) ) =0 THEN RAISE EXCEPTION
   'El ticket es obligatorio' ; END IF;
 UPDATE floatmate_objetos.viaje SET expediente=expediente || jsonb_build_object (
   'control_salida' , 'APROBADO' , 'ticket' , p_ticket )
 WHERE oid=p_viaje AND estado= 'PROGRAMADO' ;
 IF NOT FOUND THEN RAISE EXCEPTION 'No existe un viaje PROGRAMADO con ese OID' ; END IF;
END $$;
CREATE OR REPLACE PROCEDURE floatmate_objetos.sp_iniciar_viaje ( p_viaje uuid , p_salida
  timestamptz )
LANGUAGE plpgsql SET search_path TO pg_catalog AS $$
DECLARE v floatmate_objetos.viaje;
BEGIN
 SELECT b.* INTO STRICT v FROM floatmate_objetos.viaje b WHERE b.oid=p_viaje FOR UPDATE;
 IF v.estado <> 'PROGRAMADO' THEN RAISE EXCEPTION 'El viaje debe estar PROGRAMADO' ; END IF;
 UPDATE floatmate_objetos.viaje SET estado= 'EN_CURSO' , fecha_salida=p_salida WHERE
   oid=p_viaje;
 UPDATE floatmate_objetos.vehiculo SET estado= 'EN_RUTA' WHERE oid=v.vehiculo_oid;
 UPDATE floatmate_objetos.conductor SET estado= 'EN_VIAJE' WHERE oid=v.conductor_oid;
EXCEPTION WHEN no_data_found THEN RAISE EXCEPTION 'No existe el objeto viaje' ;
END $$;
CREATE OR REPLACE PROCEDURE floatmate_objetos.sp_cerrar_viaje ( p_viaje uuid , p_llegada
  timestamptz , p_km numeric )
LANGUAGE plpgsql SET search_path TO pg_catalog AS $$
DECLARE v floatmate_objetos.viaje; f floatmate_objetos.t_ficha;
BEGIN
 SELECT b.* INTO STRICT v FROM floatmate_objetos.viaje b WHERE b.oid=p_viaje FOR UPDATE;
 IF v.estado <> 'EN_CURSO' THEN RAISE EXCEPTION 'El viaje debe estar EN_CURSO' ; END IF;
 SELECT ( b.ficha ) .* INTO f FROM floatmate_objetos.vehiculo b WHERE b.oid=v.vehiculo_oid
   FOR UPDATE;
 IF p_km IS NULL OR p_km < f.kilometraje THEN RAISE EXCEPTION 'Kilometraje final invalido' ;
   END IF;
```

```
 IF p_llegada IS NULL OR p_llegada < v.fecha_salida THEN RAISE EXCEPTION
   'Fecha de llegada invalida' ; END IF;
 UPDATE floatmate_objetos.viaje SET estado= 'FINALIZADO' , fecha_llegada=p_llegada WHERE
   oid=p_viaje;
 UPDATE floatmate_objetos.vehiculo SET estado= 'DISPONIBLE' ,
   ficha=ROW ( f.marca , f.modelo , f.capacidad , p_km , f.proximo_mantenimiento_km )
     ::floatmate_objetos.t_ficha
 WHERE oid=v.vehiculo_oid;
 UPDATE floatmate_objetos.conductor SET estado= 'DISPONIBLE' WHERE oid=v.conductor_oid;
EXCEPTION WHEN no_data_found THEN RAISE EXCEPTION 'No existe el objeto viaje' ;
END $$;
COMMIT;
SELECT 'Metodos y reglas creados. Siga con 03_importar_datos_actuales.sql.' AS
  siguiente_paso;
```

Fin del script 02_metodos_y_reglas.sql

03 Cargar los datos actuales

Archivo: 03_importar_datos_actuales.sql

Qué hace: Lee el esquema floatmate y crea una instantánea de vehículos, conductores, rutas, viajes y mantenimientos en floatmate_objetos. Asigna UUID y organiza cada vehículo en su subclase.

Dónde ejecutarlo: Norte > floatmate_norte > Query Tool, después del script 02.

Cómo ejecutarlo: Ejecuta completo. Si lo repites, conserva los UUID y no duplica la carga.

Resultado esperado: Con los datos iniciales: 15 vehículos en total, 5 por subclase y 15 registros en cada una de las otras cuatro clases. La copia no se sincroniza automáticamente después.

Script completo

```
/* En la misma base que 01 y 02. Importa una instantanea de datos locales.
   No modifica ni elimina datos de floatmate. No sincroniza cambios posteriores.
   Los atributos propios de cada subclase sin equivalente en el modelo previo
   toman valores DEFAULT de demostracion: no son datos verificados de la flota.
   Reejecutar este archivo conserva los UUID y no duplica la carga. */
BEGIN;
DO $$
BEGIN
 PERFORM 1 FROM floatmate_objetos.instalacion WHERE modulo= 'BDOO' FOR UPDATE;
 IF ( SELECT datos_importados_en FROM floatmate_objetos.instalacion WHERE modulo= 'BDOO' )
   IS NOT NULL THEN
   RAISE NOTICE 'La instantanea ya fue importada; conserva sus OID y no duplica los datos.'
     ;
   RETURN;
 END IF;
 INSERT INTO floatmate_objetos.identidad_vehiculo ( terminal_codigo , id_relacional , clase
   , placa )
 SELECT terminal_codigo , vehiculo_id , tipo::floatmate_objetos.clase_vehiculo , placa
 FROM floatmate.vehiculos;
 INSERT INTO floatmate_objetos.bus_urbano ( oid , terminal_codigo , clase , placa , ficha ,
   estado )
 SELECT i.oid , v.terminal_codigo , i.clase , v.placa ,
   ROW ( v.marca , v.modelo , v.capacidad_pasajeros , v.kilometraje ,
     v.proximo_mantenimiento_km ) ::floatmate_objetos.t_ficha ,
   v.estado::floatmate_objetos.estado_vehiculo
 FROM floatmate.vehiculos v JOIN floatmate_objetos.identidad_vehiculo i
 ON i.id_relacional=v.vehiculo_id AND i.terminal_codigo=v.terminal_codigo WHERE v.tipo=
   'BUS_URBANO' ;
 INSERT INTO floatmate_objetos.bus_intermunicipal ( oid , terminal_codigo , clase , placa ,
   ficha , estado )
 SELECT i.oid , v.terminal_codigo , i.clase , v.placa ,
   ROW ( v.marca , v.modelo , v.capacidad_pasajeros , v.kilometraje ,
     v.proximo_mantenimiento_km ) ::floatmate_objetos.t_ficha ,
   v.estado::floatmate_objetos.estado_vehiculo
 FROM floatmate.vehiculos v JOIN floatmate_objetos.identidad_vehiculo i
 ON i.id_relacional=v.vehiculo_id AND i.terminal_codigo=v.terminal_codigo WHERE v.tipo=
   'BUS_INTERMUNICIPAL' ;
 INSERT INTO floatmate_objetos.microbus ( oid , terminal_codigo , clase , placa , ficha ,
   estado )
 SELECT i.oid , v.terminal_codigo , i.clase , v.placa ,
   ROW ( v.marca , v.modelo , v.capacidad_pasajeros , v.kilometraje ,
     v.proximo_mantenimiento_km ) ::floatmate_objetos.t_ficha ,
   v.estado::floatmate_objetos.estado_vehiculo
 FROM floatmate.vehiculos v JOIN floatmate_objetos.identidad_vehiculo i
 ON i.id_relacional=v.vehiculo_id AND i.terminal_codigo=v.terminal_codigo WHERE v.tipo=
```

```
    'MICROBUS' ;
  INSERT INTO floatmate_objetos.conductor ( terminal_codigo , id_relacional , documento ,
    nombre , contacto , licencia , estado )
  SELECT terminal_codigo , conductor_id , documento , concat_ws ( ' ' , nombres , apellidos )
    ,
    floatmate_objetos.fn_contacto ( telefono , NULL ) ,
    ROW ( numero_licencia , categoria_licencia , licencia_vence )
      ::floatmate_objetos.t_licencia , estado
  FROM floatmate.conductores;
  INSERT INTO floatmate_objetos.ruta ( terminal_codigo , id_relacional , nombre , origen ,
    destino , distancia_km , paradas , activa )
  SELECT terminal_codigo , ruta_id , nombre , origen , destino , distancia_km , ARRAY[origen
    , destino] , activa FROM floatmate.rutas;
  INSERT INTO floatmate_objetos.viaje ( terminal_codigo , id_relacional , vehiculo_oid ,
    conductor_oid , ruta_oid ,
    fecha_programada , fecha_salida , fecha_llegada , pasajeros , estado , expediente )
  SELECT v.terminal_codigo , v.viaje_id , ve.oid , c.oid , r.oid ,
    v.fecha_hora_programada , v.fecha_hora_salida , v.fecha_hora_llegada , v.pasajeros ,
      v.estado::floatmate_objetos.estado_viaje ,
    jsonb_strip_nulls ( jsonb_build_object ( 'observaciones' , v.observaciones ,
      'control_salida' , cs.resultado , 'ticket' , cs.numero_ticket ) )
  FROM floatmate.viajes v
  JOIN floatmate_objetos.identidad_vehiculo ve ON ve.id_relacional=v.vehiculo_id AND
    ve.terminal_codigo=v.terminal_codigo
  JOIN floatmate_objetos.conductor c ON c.id_relacional=v.conductor_id AND
    c.terminal_codigo=v.terminal_codigo
  JOIN floatmate_objetos.ruta r ON r.id_relacional=v.ruta_id AND
    r.terminal_codigo=v.terminal_codigo
  LEFT JOIN floatmate.controles_salida cs ON cs.viaje_id=v.viaje_id;
  INSERT INTO floatmate_objetos.mantenimiento ( terminal_codigo , id_relacional ,
    vehiculo_oid , tipo , fecha_ingreso , costo , detalle )
  SELECT m.terminal_codigo , m.mantenimiento_id , i.oid , m.tipo , m.fecha_ingreso , m.costo
    ,
    jsonb_build_object ( 'descripcion' , m.descripcion , 'proveedor' , m.proveedor , 'estado'
      , m.estado ,
                        'kilometraje' , m.kilometraje , 'fecha_salida' , m.fecha_salida )
  FROM floatmate.mantenimientos m JOIN floatmate_objetos.identidad_vehiculo i
  ON i.id_relacional=m.vehiculo_id AND i.terminal_codigo=m.terminal_codigo;
  UPDATE floatmate_objetos.instalacion SET datos_importados_en=CURRENT_TIMESTAMP WHERE
    modulo= 'BDOO' ;
END $$;
COMMIT;
SELECT 'Vehiculos: superclase con hijos' AS clase , count ( * ) FROM
  floatmate_objetos.vehiculo
UNION ALL SELECT 'Bus urbano' , count ( * ) FROM floatmate_objetos.bus_urbano
UNION ALL SELECT 'Bus intermunicipal' , count ( * ) FROM
  floatmate_objetos.bus_intermunicipal
UNION ALL SELECT 'Microbus' , count ( * ) FROM floatmate_objetos.microbus
UNION ALL SELECT 'Conductores' , count ( * ) FROM floatmate_objetos.conductor
UNION ALL SELECT 'Rutas' , count ( * ) FROM floatmate_objetos.ruta
UNION ALL SELECT 'Viajes' , count ( * ) FROM floatmate_objetos.viaje
UNION ALL SELECT 'Mantenimientos' , count ( * ) FROM floatmate_objetos.mantenimiento;
```

Fin del script 03_importar_datos_actuales.sql

04 Consultar y sustentar los conceptos

Archivo: 04_consultas_sustentacion.sql

Qué hace: Muestra esquemas, objetos, UUID, herencia, atributos compuestos, arreglos, métodos sobrecargados, JSON y auditoría. Estas consultas permiten explicar la implementación al profesor.

Dónde ejecutarlo: Norte > floatmate_norte > Query Tool, después del script 03.

Cómo ejecutarlo: Selecciona cada consulta y ejecútala por separado para ver su resultado. No modifica datos.

Resultado esperado: La consulta de Vehículo incluye los hijos; ONLY devuelve cero filas del padre abstracto. v_comportamiento muestra comportamientos distintos por clase.

Script completo

```
/* Cada consulta se puede seleccionar y ejecutar por separado.
   Norte > floatmate_norte > Query Tool. Solo consultas, sin modificaciones. */

-- 1. ESQUEMA: objetos de la actividad fuera de public.
SELECT table_schema , table_name , table_type FROM information_schema.tables
WHERE table_schema= 'floatmate_objetos' ORDER BY table_name;

-- 2. CLASE Y OBJETO: cada fila es una instancia persistente con OID propio.
SELECT * FROM floatmate_objetos.v_flota ORDER BY clase , placa;

-- 3. HERENCIA: SELECT sobre el padre incluye los hijos.
SELECT oid , placa , clase , tableoid::regclass AS tabla_fisica FROM
  floatmate_objetos.vehiculo ORDER BY placa;
SELECT count ( * ) AS total_con_hijos FROM floatmate_objetos.vehiculo;
SELECT count ( * ) AS solo_padre_abstracto FROM ONLY floatmate_objetos.vehiculo;
-- Esperado en la carga original: 15 y 0, respectivamente.

-- 4. EVIDENCIA TECNICA DE INHERITS: tres vinculos de hijo hacia padre.
SELECT p.relname AS superclase , h.relname AS subclase FROM pg_inherits i
JOIN pg_class p ON p.oid=i.inhparent JOIN pg_class h ON h.oid=i.inhrelid
JOIN pg_namespace n ON n.oid=p.relnamespace WHERE n.nspname= 'floatmate_objetos' ;

-- 5. TIPOS COMPUESTOS: un atributo agrupa varios datos tipados.
SELECT nombre , ( contacto ) .telefono , ( contacto ) .correo , ( licencia ) .numero , (
  licencia ) .categoria , ( licencia ) .vence
FROM floatmate_objetos.conductor ORDER BY nombre;
SELECT placa , ( ficha ) .marca , ( ficha ) .modelo , ( ficha ) .capacidad , ( ficha )
  .kilometraje
FROM floatmate_objetos.vehiculo ORDER BY placa;

-- 6. COLECCIONES: cada ruta contiene un arreglo ordenado de paradas.
SELECT nombre , paradas , paradas[1] AS primera_parada , cardinality ( paradas ) AS
  numero_paradas
FROM floatmate_objetos.ruta ORDER BY nombre;
SELECT r.nombre , p.orden , p.parada FROM floatmate_objetos.ruta r
CROSS JOIN LATERAL unnest ( r.paradas ) WITH ORDINALITY AS p ( parada , orden )
ORDER BY r.nombre , p.orden;

-- 7. IDENTIDAD: UUID del objeto y correspondencia con el ID anterior.
SELECT oid , id_relacional , terminal_codigo , clase , placa FROM
  floatmate_objetos.identidad_vehiculo ORDER BY id_relacional;
-- OID no es placa, no cambia al modificar estado y no es un OID del catalogo del motor.
-- 8. METODOS: mismo nombre y diferente comportamiento segun el tipo recibido.
```

```
SELECT floatmate_objetos.fn_describir ( b ) , floatmate_objetos.fn_costo ( b , 10::numeric )
FROM floatmate_objetos.bus_urbano b;
SELECT floatmate_objetos.fn_describir ( b ) , floatmate_objetos.fn_costo ( b , 10::numeric )
FROM floatmate_objetos.bus_intermunicipal b;
SELECT floatmate_objetos.fn_describir ( b ) , floatmate_objetos.fn_costo ( b , 10::numeric )
FROM floatmate_objetos.microbus b;
SELECT * FROM floatmate_objetos.v_comportamiento ORDER BY clase , placa;
-- Costos ficticios por 10 km: urbano 14000; intermunicipal con aire 24000; microbus 10000.

-- 9. JSON: reconstruye el objeto completo y navega hacia objetos relacionados.
SELECT jsonb_pretty ( floatmate_objetos.fn_viaje_json ( oid ) ) AS objeto_completo
FROM floatmate_objetos.viaje ORDER BY id_relacional LIMIT 1;
SELECT oid , expediente->> 'ticket' AS ticket , expediente->> 'control_salida' AS control
FROM floatmate_objetos.viaje ORDER BY id_relacional;

-- 10. METODO COMUN DE MANTENIMIENTO.
SELECT placa , floatmate_objetos.fn_km_restantes ( ficha::floatmate_objetos.t_ficha ) AS
  km_restantes
FROM floatmate_objetos.vehiculo ORDER BY placa;

-- 11. AUDITORIA / PERSISTENCIA.
SELECT evento_id , fecha , usuario , tabla , operacion , objeto_oid FROM
  floatmate_objetos.auditoria
ORDER BY evento_id DESC LIMIT 20;
SELECT * FROM floatmate_objetos.instalacion;

-- 12. INVENTARIO DE METODOS Y TRIGGERS.
SELECT p.proname AS nombre , pg_get_function_identity_arguments ( p.oid ) AS parametros ,
       CASE p.prokind WHEN 'p' THEN 'PROCEDIMIENTO' ELSE 'FUNCION' END AS tipo
FROM pg_proc p JOIN pg_namespace n ON n.oid=p.pronamespace
WHERE n.nspname= 'floatmate_objetos' ORDER BY nombre , parametros;
-- Se usa pg_trigger: information_schema repite un trigger por cada evento
-- INSERT/UPDATE/DELETE.
SELECT c.relname AS tabla , t.tgname AS trigger , pg_get_triggerdef ( t.oid ) AS definicion
FROM pg_trigger t JOIN pg_class c ON c.oid=t.tgrelid JOIN pg_namespace n ON
  n.oid=c.relnamespace
WHERE n.nspname= 'floatmate_objetos' AND NOT t.tgisinternal ORDER BY tabla , trigger;
```

Fin del script 04_consultas_sustentacion.sql

05 Probar el módulo completo

Archivo: 05_pruebas_automaticas.sql

Qué hace: Comprueba herencia, identidad, dominios, capacidad, control de salida, programación, inicio, cierre, JSON y auditoría. Captura los rechazos esperados para demostrar que las reglas funcionan.

Dónde ejecutarlo: Norte > floatmate_norte > Query Tool, después del script 03.

Cómo ejecutarlo: Ejecuta TODO el script junto. Termina con ROLLBACK y revierte los cambios del ensayo.

Resultado esperado: En Messages debe aparecer RESULTADO: todas las pruebas BDOO superadas. Requiere recursos disponibles y una licencia vigente.

Script completo

```
/* Ejecutar COMPLETO. Todas las modificaciones de demostracion terminan en ROLLBACK.
   Los errores esperados se capturan; el archivo debe finalizar correctamente.
   Leer pestaña Messages: RESULTADO: todas las pruebas BDOO superadas.
   Las secuencias de auditoria pueden dejar saltos aunque se haga ROLLBACK: es normal. */
BEGIN;
DO $$
DECLARE v floatmate_objetos.vehiculo; c floatmate_objetos.conductor; r
  floatmate_objetos.ruta;
 id uuid; anterior uuid; km numeric; n bigint; rechazo boolean; ticket text; f
   floatmate_objetos.t_ficha;
BEGIN
 IF ( SELECT datos_importados_en FROM floatmate_objetos.instalacion WHERE modulo= 'BDOO' )
   IS NULL THEN
   RAISE EXCEPTION 'Primero ejecute 03_importar_datos_actuales.sql' ;
 END IF;
 SELECT count ( * ) INTO n FROM ONLY floatmate_objetos.vehiculo;
 IF n <> 0 THEN RAISE EXCEPTION 'El padre abstracto debe estar vacio' ; END IF;
 IF ( SELECT count ( * ) FROM floatmate_objetos.vehiculo ) <> ( SELECT count ( * ) FROM
   floatmate_objetos.identidad_vehiculo ) THEN
   RAISE EXCEPTION 'Falla de identidad o herencia' ;
 END IF;
 IF EXISTS ( SELECT oid FROM floatmate_objetos.vehiculo GROUP BY oid HAVING count ( * ) <>1
   ) THEN
   RAISE EXCEPTION 'OID duplicado entre subclases' ;
 END IF;
 IF ( SELECT count ( * ) FROM floatmate_objetos.vehiculo ) < ( SELECT count ( * ) FROM
   floatmate.vehiculos )
 OR ( SELECT count ( * ) FROM floatmate_objetos.conductor ) < ( SELECT count ( * ) FROM
   floatmate.conductores )
 OR ( SELECT count ( * ) FROM floatmate_objetos.ruta ) < ( SELECT count ( * ) FROM
   floatmate.rutas )
 OR ( SELECT count ( * ) FROM floatmate_objetos.viaje ) < ( SELECT count ( * ) FROM
   floatmate.viajes )
 OR ( SELECT count ( * ) FROM floatmate_objetos.mantenimiento ) < ( SELECT count ( * ) FROM
   floatmate.mantenimientos ) THEN
   RAISE EXCEPTION
     'La instantanea no contiene todos los registros de origen o el origen cambio despues '
     'de la carga' ;
 END IF;
 IF EXISTS ( SELECT 1 FROM floatmate_objetos.bus_urbano b WHERE floatmate_objetos.fn_costo (
   b , 10::numeric ) <>14000 )
 OR EXISTS ( SELECT 1 FROM floatmate_objetos.bus_intermunicipal b WHERE
   floatmate_objetos.fn_costo ( b , 10::numeric )
   <> CASE WHEN b.aire_acondicionado THEN 24000 ELSE 22000 END )
 OR EXISTS ( SELECT 1 FROM floatmate_objetos.microbus b WHERE floatmate_objetos.fn_costo ( b
```

```
  , 10::numeric ) <>10000 ) THEN
  RAISE EXCEPTION 'Fallo en los metodos sobrecargados' ;
END IF;
SELECT b.* INTO v FROM floatmate_objetos.vehiculo b WHERE b.estado= 'DISPONIBLE' ORDER BY
  placa LIMIT 1;
IF NOT FOUND THEN RAISE EXCEPTION
  'Se requiere al menos un vehiculo DISPONIBLE para la demostracion' ; END IF;
SELECT b.* INTO c FROM floatmate_objetos.conductor b
WHERE b.estado= 'DISPONIBLE' AND b.terminal_codigo=v.terminal_codigo
  AND ( b.licencia ) .vence >= CURRENT_DATE ORDER BY documento LIMIT 1;
IF NOT FOUND THEN RAISE EXCEPTION 'Se requiere conductor disponible con licencia vigente' ;
  END IF;
SELECT b.* INTO r FROM floatmate_objetos.ruta b WHERE b.activa AND
  b.terminal_codigo=v.terminal_codigo ORDER BY nombre LIMIT 1;
IF NOT FOUND THEN RAISE EXCEPTION 'Se requiere ruta activa' ; END IF;

rechazo:=false;
BEGIN UPDATE floatmate_objetos.vehiculo SET oid=gen_random_uuid ( ) WHERE oid=v.oid;
EXCEPTION WHEN raise_exception THEN rechazo:=true; END;
IF NOT rechazo THEN RAISE EXCEPTION 'No se protegio el OID' ; END IF;
RAISE NOTICE 'OK: identidad inmutable' ;

rechazo:=false;
BEGIN PERFORM ROW ( 'Marca' , 2020::smallint , 0::smallint , 0::numeric , 5000::numeric )
  ::floatmate_objetos.t_ficha::floatmate_objetos.d_ficha;
EXCEPTION WHEN check_violation THEN rechazo:=true; END;
IF NOT rechazo THEN RAISE EXCEPTION 'No se valido el tipo compuesto' ; END IF;
RAISE NOTICE 'OK: dominio rechaza capacidad invalida' ;

rechazo:=false;
BEGIN INSERT INTO floatmate_objetos.vehiculo SELECT v.*;
EXCEPTION WHEN raise_exception THEN rechazo:=true; END;
IF NOT rechazo THEN RAISE EXCEPTION 'Se permitio instanciar el padre' ; END IF;
RAISE NOTICE 'OK: superclase abstracta' ;

rechazo:=false;
BEGIN
  INSERT INTO floatmate_objetos.identidad_vehiculo ( terminal_codigo , clase , placa )
  VALUES ( v.terminal_codigo , v.clase , 'ZZZ999' ) ;
  SET CONSTRAINTS ALL IMMEDIATE;
EXCEPTION WHEN raise_exception THEN rechazo:=true; END;
IF NOT rechazo THEN RAISE EXCEPTION 'Se permitio una identidad sin objeto' ; END IF;
SET CONSTRAINTS ALL DEFERRED;
RAISE NOTICE 'OK: identidad requiere instancia' ;

rechazo:=false;
BEGIN
 CALL floatmate_objetos.sp_programar_viaje ( v.oid , c.oid , r.oid ,
   CURRENT_TIMESTAMP+interval '10 years' , 81::smallint ) ;
EXCEPTION WHEN raise_exception THEN rechazo:=true; END;
IF NOT rechazo THEN RAISE EXCEPTION 'No se bloqueo el sobrecupo' ; END IF;
RAISE NOTICE 'OK: sobrecupo rechazado' ;

CALL floatmate_objetos.sp_programar_viaje ( v.oid , c.oid , r.oid ,
  CURRENT_TIMESTAMP+interval '1 day' , 10::smallint ) ;
SELECT b.oid INTO id FROM floatmate_objetos.viaje b WHERE b.id_relacional IS NULL
AND b.vehiculo_oid=v.oid AND b.fecha_programada=CURRENT_TIMESTAMP+interval '1 day' ;
IF id IS NULL THEN RAISE EXCEPTION 'No se creo el objeto viaje' ; END IF;

rechazo:=false;
BEGIN CALL floatmate_objetos.sp_iniciar_viaje ( id , CURRENT_TIMESTAMP+interval '1 day' ) ;
EXCEPTION WHEN raise_exception THEN rechazo:=true; END;
```

```
  IF NOT rechazo THEN RAISE EXCEPTION 'Se inicio sin control' ; END IF;
  RAISE NOTICE 'OK: inicio sin control rechazado' ;

  CALL floatmate_objetos.sp_aprobar_control ( id , 'DEMO-BDOO-001' ) ;
  CALL floatmate_objetos.sp_iniciar_viaje ( id , CURRENT_TIMESTAMP+interval '1 day' ) ;
  IF NOT EXISTS ( SELECT 1 FROM floatmate_objetos.vehiculo b WHERE b.oid=v.oid AND b.estado=
    'EN_RUTA' ) THEN
    RAISE EXCEPTION 'No se actualizo el vehiculo' ;
  END IF;
  IF NOT EXISTS ( SELECT 1 FROM floatmate_objetos.conductor b WHERE b.oid=c.oid AND b.estado=
    'EN_VIAJE' ) THEN
    RAISE EXCEPTION 'No se actualizo el conductor' ;
  END IF;
  IF floatmate_objetos.fn_viaje_json ( id ) -> 'vehiculo' ->> 'oid' IS DISTINCT FROM
    v.oid::text THEN
    RAISE EXCEPTION 'Falla en la navegacion de objetos JSON' ;
  END IF;
  km:= ( v.ficha ) .kilometraje+50;
  CALL floatmate_objetos.sp_cerrar_viaje ( id , CURRENT_TIMESTAMP+interval '1 day 2 hours' ,
    km ) ;
  IF NOT EXISTS ( SELECT 1 FROM floatmate_objetos.viaje b WHERE b.oid=id AND b.estado=
    'FINALIZADO' )
  OR NOT EXISTS ( SELECT 1 FROM floatmate_objetos.vehiculo b WHERE b.oid=v.oid AND b.estado=
    'DISPONIBLE' AND ( b.ficha ) .kilometraje=km )
  OR NOT EXISTS ( SELECT 1 FROM floatmate_objetos.conductor b WHERE b.oid=c.oid AND b.estado=
    'DISPONIBLE' ) THEN
    RAISE EXCEPTION 'Fallo de cierre del viaje' ;
  END IF;
  IF NOT EXISTS ( SELECT 1 FROM floatmate_objetos.auditoria a WHERE a.objeto_oid=v.oid AND
    a.operacion= 'UPDATE' AND a.anterior IS NOT NULL AND a.nuevo IS NOT NULL ) THEN
    RAISE EXCEPTION 'No se registro auditoria' ;
  END IF;
  SET CONSTRAINTS ALL IMMEDIATE;
  RAISE NOTICE 'OK: programacion, control, inicio, cierre, JSON, auditoria y OID estable' ;
  RAISE NOTICE 'RESULTADO: todas las pruebas BDOO superadas. Ahora se hace ROLLBACK.' ;
END $$;
ROLLBACK;
SELECT 'Pruebas terminadas. Los datos de demostracion fueron revertidos.' AS resultado;
```

Fin del script 05_pruebas_automaticas.sql

06 Consultar datos distribuidos como tipos compuestos

Archivo: 06_puente_bdd_opcional.sql

Qué hace: Crea vistas en vivo que representan fichas y contactos de Norte y Sur como valores compuestos, utilizando las vistas globales de la actividad BDD anterior.

Dónde ejecutarlo: Solo Norte > floatmate_norte > Query Tool. Requiere floatmate_global y FDW funcionando.

Cómo ejecutarlo: Ejecuta completo. Es opcional; no se necesita para instalar el módulo principal.

Resultado esperado: Con las cargas iniciales se consultan 15 vehículos NOR y 15 SUR. Estas vistas no asignan UUID persistentes a Sur ni federan los objetos nuevos de ese nodo.

Script completo

```
/* OPCIONAL. Solo NORTE > floatmate_norte. Requiere la federacion anterior funcionando.
   Proyecciones en vivo sobre floatmate_global; no crean copias ni OID persistentes de Sur.
   Sirve para mostrar que BDD (ubicacion) y BDOO (representacion) pueden coexistir. */
BEGIN;
DO $$
BEGIN
 IF to_regclass ( 'floatmate_global.vehiculos' ) IS NULL OR to_regclass (
   'floatmate_global.conductores' ) IS NULL THEN
   RAISE EXCEPTION
     'Primero configure la federacion anterior y sus vistas globales; este archivo es '
     'opcional.' ;
 END IF;
END $$;
CREATE OR REPLACE VIEW floatmate_objetos.v_ficha_global_en_vivo AS
 SELECT v.terminal_codigo , v.vehiculo_id AS id_relacional , v.nodo_origen , v.placa ,
   v.tipo ,
   ROW ( v.marca , v.modelo , v.capacidad_pasajeros , v.kilometraje ,
     v.proximo_mantenimiento_km ) ::floatmate_objetos.t_ficha AS ficha ,
   v.estado
 FROM floatmate_global.vehiculos v;
CREATE OR REPLACE VIEW floatmate_objetos.v_contacto_global_en_vivo AS
 SELECT c.terminal_codigo , c.conductor_id AS id_relacional , c.nodo_origen ,
   concat_ws ( ' ' , c.nombres , c.apellidos ) AS nombre ,
   ROW ( c.telefono , NULL::text ) ::floatmate_objetos.t_contacto AS contacto ,
   ROW ( c.numero_licencia , c.categoria_licencia , c.licencia_vence )
     ::floatmate_objetos.t_licencia AS licencia
 FROM floatmate_global.conductores c;
COMMIT;
SELECT terminal_codigo , nodo_origen , count ( * ) AS vehiculos
FROM floatmate_objetos.v_ficha_global_en_vivo GROUP BY terminal_codigo , nodo_origen;
SELECT terminal_codigo , placa , ( ficha ) .marca , ( ficha ) .capacidad
FROM floatmate_objetos.v_ficha_global_en_vivo ORDER BY terminal_codigo , placa;
/* Con los datos iniciales y FDW conectado: NOR=15, SUR=15.
   Estas vistas no federan los UUID del modulo de Sur: federan el modelo previo. */
```

Fin del script 06_puente_bdd_opcional.sql

07 Demostrar el ciclo de un viaje

Archivo: 07_demostracion_guiada.sql

Qué hace: Permite mostrar la creación de un viaje, aprobación de control, inicio, cierre, auditoría y rechazo del cambio de UUID. Elige recursos automáticamente.

Dónde ejecutarlo: Norte > floatmate_norte > Query Tool, con el módulo instalado y los datos cargados.

Cómo ejecutarlo: Ejecuta sus SEIS SECCIONES una por una, en orden, en la MISMA pestaña. La última hace ROLLBACK.

Resultado esperado: Puedes mostrar los estados durante cada paso. Al ejecutar la última sección desaparecen los cambios del ensayo.

Script completo

```
/* DEMOSTRACION MANUAL EN PGADMIN, NO EJECUTAR TODAS LAS PARTES A LA VEZ.
   Seleccionar una seccion y pulsar F5. Hacerlo todo en la MISMA pestana Query Tool.
   La ultima seccion revierte los cambios. El error esperado de la seccion 5 esta capturado.
      */

-- SECCION 1: abrir una transaccion y elegir recursos existentes automaticamente.
BEGIN;
CREATE TEMP TABLE demo_bdoo_recursos ON COMMIT DROP AS
SELECT v.oid AS vehiculo , c.oid AS conductor , r.oid AS ruta , ( v.ficha ) .kilometraje AS
  km
FROM floatmate_objetos.vehiculo v
JOIN floatmate_objetos.conductor c ON c.terminal_codigo=v.terminal_codigo
JOIN floatmate_objetos.ruta r ON r.terminal_codigo=v.terminal_codigo
WHERE v.estado= 'DISPONIBLE' AND c.estado= 'DISPONIBLE' AND r.activa
AND ( c.licencia ) .vence >= ( CURRENT_TIMESTAMP+interval '1 day' ) ::date
ORDER BY v.placa , c.documento , r.nombre LIMIT 1;
SELECT * FROM demo_bdoo_recursos;

-- SECCION 2: invocar el metodo para crear un objeto viaje.
DO $$
DECLARE p record;
BEGIN
 SELECT * INTO p FROM demo_bdoo_recursos;
 IF NOT FOUND THEN RAISE EXCEPTION 'No hay recursos disponibles para la demostracion' ; END
   IF;
 CALL floatmate_objetos.sp_programar_viaje ( p.vehiculo , p.conductor , p.ruta ,
   CURRENT_TIMESTAMP+interval '1 day' , 10::smallint ) ;
END $$;
CREATE TEMP TABLE demo_bdoo_viaje ON COMMIT DROP AS
SELECT v.oid FROM floatmate_objetos.viaje v JOIN demo_bdoo_recursos d ON
  d.vehiculo=v.vehiculo_oid
WHERE v.id_relacional IS NULL AND v.fecha_programada=CURRENT_TIMESTAMP+interval '1 day' ;
SELECT v.oid , v.estado FROM floatmate_objetos.viaje v JOIN demo_bdoo_viaje d USING ( oid )
  ;

-- SECCION 3: registrar control e iniciar. El metodo cambia tres objetos.
DO $$
DECLARE id uuid;
BEGIN
 SELECT oid INTO STRICT id FROM demo_bdoo_viaje;
 CALL floatmate_objetos.sp_aprobar_control ( id , 'TICKET-DEMO-BDOO' ) ;
 CALL floatmate_objetos.sp_iniciar_viaje ( id , CURRENT_TIMESTAMP+interval '1 day' ) ;
END $$;
```

```
SELECT jsonb_pretty ( floatmate_objetos.fn_viaje_json ( oid ) ) FROM demo_bdoo_viaje;

-- SECCION 4: finalizar y comprobar que los OID originales permanecen.
DO $$
DECLARE id uuid; km numeric;
BEGIN
 SELECT oid INTO STRICT id FROM demo_bdoo_viaje;
 SELECT d.km+50 INTO km FROM demo_bdoo_recursos d;
 CALL floatmate_objetos.sp_cerrar_viaje ( id , CURRENT_TIMESTAMP+interval '1 day 2 hours' ,
   km ) ;
END $$;
SELECT v.oid , v.estado , ( v.ficha ) .kilometraje FROM floatmate_objetos.vehiculo v
JOIN demo_bdoo_recursos d ON d.vehiculo=v.oid;
SELECT v.oid , v.estado FROM floatmate_objetos.viaje v JOIN demo_bdoo_viaje d USING ( oid )
  ;
SELECT operacion , tabla , objeto_oid , anterior , nuevo FROM floatmate_objetos.auditoria
WHERE objeto_oid IN ( SELECT oid FROM demo_bdoo_viaje ) ORDER BY evento_id;

-- SECCION 5: demostrar que el OID no se puede modificar; el error queda capturado.
DO $$
BEGIN
 BEGIN
   UPDATE floatmate_objetos.viaje SET oid=gen_random_uuid ( ) WHERE oid IN ( SELECT oid FROM
     demo_bdoo_viaje ) ;
 EXCEPTION WHEN raise_exception THEN RAISE NOTICE 'RECHAZO ESPERADO: %' , SQLERRM;
 END;
END $$;

-- SECCION 6: revertir TODO el ensayo y salir de la transaccion.
ROLLBACK;
SELECT 'Demostracion finalizada: cambios revertidos y OID de la carga conservados.' AS
  resultado;
```

Fin del script 07_demostracion_guiada.sql

08 Crear un vehículo de ejemplo

Archivo: 08_crear_objeto_ejemplo.sql

Qué hace: Construye temporalmente la identidad UUID de un vehículo y su instancia BusUrbano, con ficha, equipamiento y atributos propios. Después consulta sus métodos.

Dónde ejecutarlo: Norte > floatmate_norte > Query Tool, con el módulo instalado.

Cómo ejecutarlo: Ejecuta completo. Termina con ROLLBACK. Utiliza la placa de demostración ZZZ998.

Resultado esperado: Se ve el objeto nuevo, su UUID y sus atributos. Después del ROLLBACK el objeto de ejemplo desaparece; la carga inicial permanece.

Script completo

```
/* EJEMPLO ADICIONAL DE CREACION / CONSTRUCCION DE UN OBJETO NUEVO.
   Ejecutar completo en Norte > floatmate_norte. Termina con ROLLBACK.
   Usa placa de demostracion: si ZZZ998 ya existe, cambiela por una libre.
   Para guardar un objeto real se confirmaria la transaccion con COMMIT,
   despues de proporcionar sus datos reales y revisar el resultado. */
BEGIN;
DO $$
DECLARE id uuid;
BEGIN
 INSERT INTO floatmate_objetos.identidad_vehiculo ( terminal_codigo , clase , placa )
 VALUES ( 'NOR' , 'BUS_URBANO' , 'ZZZ998' ) RETURNING oid INTO id;
 INSERT INTO floatmate_objetos.bus_urbano ( oid , terminal_codigo , placa , ficha , estado ,
   equipamiento , puertas , accesible )
 VALUES ( id , 'NOR' , 'ZZZ998' ,
   ROW ( 'Marca de demostracion' , 2024::smallint , 40::smallint , 1000::numeric ,
     10000::numeric ) ::floatmate_objetos.t_ficha ,
   'DISPONIBLE' , ARRAY[ 'BOTIQUIN' , 'EXTINTOR' , 'GPS' ] , 2 , true ) ;
 RAISE NOTICE 'Objeto creado temporalmente: UUID %' , id;
END $$;
-- Debe incluir la nueva instancia, sus atributos y su comportamiento.
SELECT b.oid , b.placa , ( b.ficha ) .capacidad , b.equipamiento ,
 floatmate_objetos.fn_describir ( b ) AS descripcion , floatmate_objetos.fn_costo ( b ,
   10::numeric ) AS costo
FROM floatmate_objetos.bus_urbano b WHERE placa= 'ZZZ998' ;
SET CONSTRAINTS ALL IMMEDIATE;
ROLLBACK;
SELECT 'Objeto de demostracion revertido. La carga inicial se conserva.' AS resultado;
```

Fin del script 08_crear_objeto_ejemplo.sql